

FATECS - CENTRO UNIVERSITARIO DO DISTRITO FEDERAL

Teoria dos Compiladores

Trabalho 2 - Analisador Sintatico (Parser)

Java-- com JFlex + JCup

Aluno: Joao Vitor Alecrim

Professora: Alessandra Hauck

Brasilia, junho de 2026

1. Introducao

Este trabalho implementa um Analisador Sintatico (Parser) para um subconjunto da linguagem Java denominado **Java--**. O analisador foi desenvolvido utilizando as ferramentas **JFlex** (gerador de scanners/analísadores lexicos) e **JCup** (gerador de parsers LALR(1)), integrando as duas etapas da analise em um pipeline completo.

O scanner reconhece os tokens da linguagem Java-- incluindo palavras reservadas, identificadores, literais numericos (inteiros, reais e hexadecimais), constantes de caractere, operadores e delimitadores, reportando erros lexicos para construcoes invalidas. O parser valida a estrutura sintatica do programa seguindo uma gramatica LALR(1) nao ambigua.

2. Estrutura dos Arquivos

Arquivo	Descricao
Scanner.flex	Especificacao JFlex do analisador lexico. Define tokens, macros e regras de reconhecimento.
Scanner.java	Codigo Java gerado automaticamente pelo JFlex a partir de Scanner.flex.
Parser.cup	Especificacao JCup da gramatica Java--. Define terminais, nao-terminais e producoes.
parser.java	Codigo Java do parser LALR(1) gerado automaticamente pelo JCup.
sym.java	Classe de constantes dos simbolos terminais, gerada automaticamente pelo JCup.
Main.java	Programa principal. Instancia o Scanner e o Parser e inicia a analise.
MainExercicio.java	Main alternativo para os exercicios de expressoes aritmeticas (Aulas 14-17).
exercicio.flex	Scanner para expressoes aritmeticas - Exercicios Aulas 14-15 (com valores Double).
exercicio.cup	Parser para expressoes aritmeticas com Traducao Dirigida por Sintaxe (RESULT).
ex4.cup	Variante do exercicio.cup com verificacao de denominador zero.
exercicio17.flex	Scanner da Aula 17: adiciona IDENT, DOT, COL_ABRE, COL_FECHA.
exercicio17.cup	Parser da Aula 17: adiciona producao designator (variavel, obj.attr, arr[i]).
entrada.txt	Arquivo de entrada com programa Java-- valido para teste.
entrada_erros.txt	Arquivo de entrada com erros lexicos para demonstracao.
saida.txt	Saida da execucao sobre entrada.txt (analise bem-sucedida).
saida_erros.txt	Saida da execucao sobre entrada_erros.txt (com erros lexicos).

3. Analisador Lexico (Scanner)

O scanner foi implementado com JFlex e e capaz de reconhecer todos os tokens da linguagem Java--. O arquivo **Scanner.flex** esta configurado com as diretivas %cup (integracao com JCup), %public (construtor publico) e %unicode.

3.1 Tokens Reconhecidos

Categoria	Tokens / Exemplos
Palavras reservadas	program final class void if else while return read print new int float cha
Identificadores	IDENT = letra (letra digito _)* Ex: x media Ponto calcular
Inteiros	INT_LIT = 0 [1-9][0-9]* Ex: 0 42 100
Reais	FLOAT_LIT = digito+ "." digito+ Ex: 3.14 0.5 100.0
Hexadecimais	HEX_LIT = 0x[0-9A-Fa-f]+ Ex: 0xFF 0x1A 0xDEAD
Char constante	CHAR_CONST = 'c' (com escapes: \n \t \r \b \f \' \" \\)
Operadores rel.	== != >= <= > < =
Operadores arit.	+ - * / %
Delimitadores	{ } () [] ; , .
Comentarios	/* ... */ (nao aninhado - detecta e reporta aninhamento)

3.2 Erros Lexicos Detectados

O scanner detecta e reporta os seguintes erros lexicos, continuando a analise apos cada erro:

Construcao Invalida	Exemplo	Mensagem Gerada
Hex com X maiusculo	0XFF	[linha,col] erro lexico: hexadecimal invalido -> 0XFF
Hex incompleto	0x	[linha,col] erro lexico: hexadecimal incompleto -> 0x
Real sem parte decimal	5.	[linha,col] erro lexico: numero real invalido -> 5.
Real sem parte inteira	.5	[linha,col] erro lexico: numero real invalido -> .5
Comentario nao fechado	/* ...	[linha,col] erro lexico: comentario nao fechado
Comentario aninhado	/* /* */	[linha,col] erro lexico: comentario aninhado nao permitido
Caracter desconhecido	@ \$ #	[linha,col] erro lexico: <char>

4. Analisador Sintatico (Parser)

O parser foi implementado com JCup, gerando um analisador LALR(1). A gramatica cobre a estrutura completa da linguagem Java--, desde declaracoes de programa e classes ate expressoes aritmeticas e estruturas de controle.

4.1 Estrutura de um Programa Java--

```
program <nome>

    [const_decl | var_decl | class_decl]*

{
    [method_decl]*
}
```

Exemplo de programa valido:

```
program Exemplo

class Ponto { int x; int y; }
final int MAX = 100;
int contador;

{
    int soma(int a, int b) {
        return a + b;
    }

    void main() {
        int x;
        x = 10;
        if (x > 0) { x = x + 1; } else { x = x - 1; }
        while (x < MAX) { x = x + 1; }
        read(x);
        print(x, 5);
        return;
    }
}
```

4.2 Gramatica (resumo das principais producoes)

Nao-terminal	Producoes
program	PROGRAM IDENT decl_list { method_decl_list }
decl_list	vazio decl_list const_decl decl_list var_decl decl_list class_decl
const_decl	final type IDENT = (number charConst) ;
var_decl	type IDENT {, IDENT} ;
class_decl	class IDENT { var_decl_list }
method_decl	(type void) IDENT ([FormPars]) var_decl_list block
type	int float char IDENT (opcionalmente com [])

statement	Designator = Expr ; if (Cond) Stmt [else Stmt] while (Cond) Stmt return [Expr] ; read (Designator) ; print (Expr [, NUM_INT]) ; Block
expr	[-] term { (+ -) term }
term	factor { (* / %) factor }
factor	Designator [ActPars] number charConst new IDENT [[Expr]] (Expr)
designator	IDENT { . IDENT [Expr] }
condition	Expr relop Expr (relop: == != > >= < <=)

Conflito Shift/Reduce: A gramática possui um conflito shift/reduce clássico no if-then-else (dangling else). O Jcup resolve automaticamente em favor do *shift*, associando o else ao if mais próximo, que é o comportamento correto. O comando `-expect 1` é usado para aceitar este conflito esperado.

5. Exercicios de Expressoes Aritmeticas (Aulas 14-17)

Alem do parser Java--, o projeto inclui os exercicios desenvolvidos durante as Aulas 14 a 17, que implementam progressivamente um reconhecedor de expressoes aritmeticas com Traducaao Dirigida por Sintaxe.

Arquivo	Aula	Funcionalidade
exercicio.flex exercicio.cup	14-15	Expressoes aritmeticas com +, -, *, /, %, () Tokens tipados (Double). Acoes semanticas com RESULT. Imprime "Resultado = <valor>" para cada expressao.
ex4.cup	15	Igual ao exercicio.cup mas com verificacao de denominador zero em / e %. Imprime "Error - Denominador invalido."
exercicio17.flex exercicio17.cup	17	Adiciona IDENT (identificadores) e DESIGNATOR. Designator: variavel, obj.attr, arr[i], obj.arr[i].attr

5.1 Gramatica dos Exercicios (exercicio.cup)

```
terminal Double INTEIRO;  
terminal String IDENT;          /* Aula 17 */  
terminal MAIS, MENOS, VEZES, DIVISAO, MOD, ABRE, FECHA;  
terminal DOT, COL_ABRE, COL_FECHA; /* Aula 17 */  
  
non terminal Double expr_list, expr_ptv, expr, term, factor;  
non terminal String designator; /* Aula 17 */  
  
expr_ptv ::= expr:e PTVIRG  {: System.out.println("Resultado = " + e); :} ;  
expr      ::= expr:e MAIS term:t    {: RESULT = e + t; :}  
          | expr:e MENOS term:t    {: RESULT = e - t; :}  
          | term:t                  {: RESULT = t; :} ;  
term      ::= term:t VEZES factor:f {: RESULT = t * f; :}  
          | term:t DIVISAO factor:f {: RESULT = t / f; :}  
          | term:t MOD factor:f    {: RESULT = t % f; :}  
          | factor:f                {: RESULT = f; :} ;  
factor    ::= INTEIRO:n {: RESULT = n; :}  
          | ABRE expr:e FECHA {: RESULT = e; :}  
          | designator:d {: RESULT = null; :} ; /* Aula 17 */  
designator ::= IDENT:id          {: RESULT = id; :}  
          | designator:d DOT IDENT:id    {: RESULT = d+"."+id; :}  
          | designator:d COL_ABRE expr COL_FECHA {: RESULT = d+"[...]"; :} ;
```

6. Como Compilar e Executar

Todos os comandos devem ser executados no PowerShell dentro da pasta **TrabalhoScannerJFlex** (ou da pasta **bin** se os JARs estiverem lá).

6.1 Pipeline - Parser Java-- (scanner1_cup.flex + parser1.cup)

```
# 1. Gerar parser.java e sym.java (JCup - sempre primeiro)
java -jar .\jflex-1.9.1\bin\java-cup-11b.jar -expect 1 .\Parser.cup

# 2. Gerar Scanner.java (JFlex)
.\jflex-1.9.1\bin\jflex.bat .\Scanner.flex

# 3. Compilar todos os .java
javac -cp .;\jflex-1.9.1\bin\java-cup-11b-runtime.jar *.java

# 4. Executar - entrada valida
java -cp .;\jflex-1.9.1\bin\java-cup-11b-runtime.jar Main .\entrada.txt

# 4b. Executar - entrada com erros
java -cp .;\jflex-1.9.1\bin\java-cup-11b-runtime.jar Main .\entrada_erros.txt
```

6.2 Pipeline - Exercicios de Expressoes (exercicio.flex + exercicio.cup)

```
# 1. JCup
java -jar .\jflex-1.9.1\bin\java-cup-11b.jar .\exercicio.cup

# 2. JFlex
.\jflex-1.9.1\bin\jflex.bat .\exercicio.flex

# 3. Compilar
javac -cp .;\jflex-1.9.1\bin\java-cup-11b-runtime.jar *.java

# 4. Executar
java -cp .;\jflex-1.9.1\bin\java-cup-11b-runtime.jar MainExercicio .\teste_ex.txt
```

6.3 Exercicio 4 (ex4.cup - denominador zero)

```
java -jar .\jflex-1.9.1\bin\java-cup-11b.jar .\ex4.cup
.\jflex-1.9.1\bin\jflex.bat .\exercicio.flex # mesmo scanner
javac -cp .;\jflex-1.9.1\bin\java-cup-11b-runtime.jar *.java
java -cp .;\jflex-1.9.1\bin\java-cup-11b-runtime.jar MainExercicio .\teste_ex.txt
```

7. Casos de Teste e Saidas

7.1 Parser Java-- - Entrada Valida (entrada.txt)

```
program Compiladores
class Ponto { int x; int y; }
int contador;
final int MAX = 100;
{
    int soma(int a, int b) { return a + b; }
    void main() {
        int x;
        x = 10;
        x = 0xFF;
        if (x > 0) { x = x + 1; } else { x = x - 1; }
        while (x < 100) { x = x + 1; }
        read(x);
        print(x, 5);
        return;
    }
}
```

Saida esperada:

Analise sintatica concluida com sucesso.

7.2 Parser Java-- - Entrada com Erros (entrada_erros.txt)

```
program Teste
{
    void main() {
        int x;
        x = 0xFF;    /* ERRO: hex invalido */
        x = 5.;      /* ERRO: real invalido */
        x = .5;      /* ERRO: real invalido */
        x = 0x;      /* ERRO: hex incompleto */
        x = 42;
        print(x);
        return;
    }
}
```

Saida esperada:

[8,9] erro lexico: hexadecimal invalido -> 0xFF
[9,9] erro lexico: numero real invalido -> 5.
[10,9] erro lexico: numero real invalido -> .5
[11,9] erro lexico: hexadecimal incompleto -> 0x
Analise sintatica concluida com sucesso.

7.3 Exercicios de Expressoes - teste_ex.txt

```
1+2-3;
(3+7)*2;
10/(2+3);
```

Saida esperada (exercicio.cup):

Resultado = 0.0
Resultado = 20.0

Resultado = 2.0

7.4 Exercício 4 - Verificação de Denominador

```
10/0;
```

Saida esperada (ex4.cup):

```
Error - Denominador invalido.  
Resultado = null
```

8. Conclusao

O trabalho implementa com sucesso o pipeline completo de analise lexico-sintatica para a linguagem Java--. O scanner, gerado com JFlex, reconhece todos os tokens da linguagem e reporta erros lexicos com informacao de linha e coluna. O parser, gerado com JCup, valida a estrutura sintatica segundo uma gramatica LALR(1) cobrindo declaracoes, metodos, expressoes e estruturas de controle.

Complementarmente, os exercicios das Aulas 14 a 17 demonstram a aplicacao de Traducao Dirigida por Sintaxe, adicionando acoes semanticas (blocos `{: :}`) e atributos sintetizados (RESULT) para calcular e imprimir resultados de expressoes aritmeticas, alem de estender a gramatica com suporte a identificadores e designators (variaveis, atributos de objetos e acesso a vetores).

O projeto atende aos requisitos da disciplina e serve como base para as proximas etapas da implementacao de um compilador completo.